

ГЛАВА 3. РАЗДЕЛ 3.1 

Введение в SQL

От теории реляционных моделей к практическому управлению
данными

| Что такое SQL и зачем он нужен? 🤔

SQL (Structured Query Language) — это стандартизированный язык структурированных запросов для работы с базами данных.

- ✓ **Специализация:** Это язык работы с данными, а не общего программирования.
- ✓ **Основные задачи:** Создание таблиц, наполнение, обновление, удаление, выборка и анализ данных.
- ✓ **Управление доступом:** Обеспечение безопасности СУБД.



| Где используется SQL?

Язык SQL является универсальным стандартом и поддерживается всеми популярными реляционными системами управления базами данных (СУБД).



PostgreSQL

Продвинутая объектно-реляционная СУБД с открытым исходным кодом. Идеальна для сложных систем.



MySQL

Самая популярная СУБД для веб-разработки и средних проектов. Простая в настройке и надежная.



Oracle & MS SQL

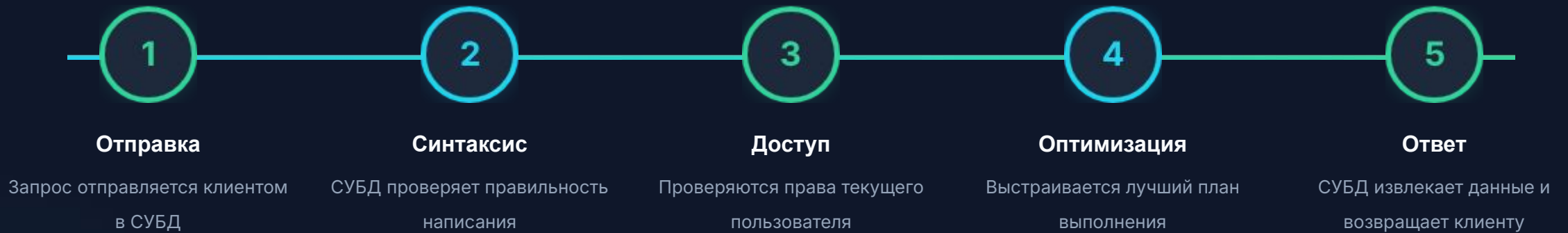
Корпоративные гиганты для сверхбольших нагрузок, критически важных транзакций и аналитики.



SQLite

Встраиваемая легковесная база данных. Используется в смартфонах, браузерах и локальном ПО.

| Как SQL взаимодействует с СУБД? ⚡



Основные группы SQL-команд



DDL

Data Definition Language

Определение структуры базы данных. Создание, изменение, удаление таблиц.

CREATE, ALTER, DROP



DML

Data Manipulation Language

Манипулирование данными. Наполнение таблиц, обновление строк и удаление.

INSERT, UPDATE, DELETE



DQL

Data Query Language

Язык запросов. Извлечение необходимых данных согласно условиям.

SELECT



DCL

Data Control Language

Управление доступом к данным. Назначение и отзыв прав пользователей.

GRANT, REVOKE

Создание первой таблицы (CREATE TABLE)

```
students.sql

CREATE TABLE Students (
  id INT PRIMARY KEY,
  name VARCHAR(50),
  age INT
);
```

Разбор синтаксиса:

- **CREATE TABLE Students** — инициирует создание таблицы с именем Students.
- **id INT PRIMARY KEY** — уникальное целочисленное поле (первичный ключ).
- **name VARCHAR(50)** — текстовое поле (строка до 50 символов).
- **age INT** — целочисленное поле для хранения возраста.

Вставка данных в таблицу (INSERT)

Одиночная вставка

 insert_one.sql

```
INSERT INTO Students (id, name, age)
VALUES (1, 'Ivan', 19);
```

Мы указываем таблицу, перечисляем поля в скобках, а затем передаем соответствующие значения после ключевого слова **VALUES**.

Множественная вставка

 insert_many.sql

```
INSERT INTO Students (id, name, age)
VALUES
(2, 'Anna', 20),
(3, 'Petr', 21);
```

Чтобы вставить сразу несколько строк, разделяйте кортежи значений запятыми. Это работает значительно быстрее.

| Чтение и фильтрация данных (SELECT)

1. Чтение всех полей

   select_all.sql

```
SELECT * FROM Students;
```

Символ `*` означает выборку абсолютно всех колонок из таблицы.

2. Выборочные колонки

   select_columns.sql

```
SELECT name, age FROM Students;
```

3. Фильтрация строк (WHERE)

   select_where.sql

```
SELECT * FROM Students  
WHERE age = 19;
```

Конструкция позволяет выводить только те строки, которые соответствуют условию.

4. Сложное условие

   select_complex.sql

```
SELECT * FROM Students  
WHERE age > 18;
```

Безопасное обновление и удаление ⚠

Обновление данных (UPDATE)

● ● ● update.sql

```
UPDATE Students  
SET age = 20  
WHERE id = 1;
```

Конструкция **WHERE** критически важна! Без нее СУБД обновит возраст **каждого** студента в таблице.

Удаление данных (DELETE)

● ● ● delete.sql

```
DELETE FROM Students  
WHERE id = 3;
```

Без оператора **WHERE** команда **DELETE** полностью очистит таблицу от записей!



Золотое правило работы в СУБД

Никогда не выполняйте UPDATE и DELETE в производственной базе данных без предварительной проверки условий WHERE.

Философия SQL: Декларативный подход

Главное отличие SQL от классического программирования — его **декларативный характер**.

«Мы описываем результат, который хотим получить (ЧТО сделать), а СУБД сама планирует оптимальный путь решения (КАК сделать)».

- **Универсальность:** Работает с огромными массивами данных одинаково эффективно.
- **Простота:** Легко читать и поддерживать структуру запроса.
- **Энергоэффективность:** Оптимизатор СУБД делает всю тяжелую работу за разработчика.

Императивный vs Декларативный подход

Императивный подход (JS/Python)

```
imperative.py

result = []
for student in students:
    if student.age > 18:
        result.append(student)
# Описываем каждый шаг алгоритма
```

Мы пишем полный цикл, вручную проверяем условия и собираем результирующий список.

Декларативный подход (SQL)

```
declarative.sql

SELECT * FROM Students
WHERE age > 18;

# Просто описываем желаемый результат
```

Мы не заботимся об организации циклов. СУБД самостоятельно находит самый быстрый способ фильтрации.



Подведем итоги

SQL — фундаментальный инструмент для взаимодействия со всеми реляционными базами данных.

В следующей лекции мы детально изучим оператор **SELECT**, научимся сортировать, группировать и объединять таблицы!

Спасибо за внимание! 

| Image Sources



https://static.vecteezy.com/system/resources/previews/010/354/683/large_2x/panoramic-view-of-multiple-database-with-relational-database-tables-background-concept-of-database-server-sql-data-storage-data-center-webhosting-3d-illustration-photo.jpg

Source: www.vecteezy.com